

PRODUCT REQUIREMENTS DOCUMENT

InterviewAI

AI-Powered Mock Interview Platform

Version 1.0

Date: July 2026

Owner: Ayush Sharma

Repository: ayushsharma121996-sys/AI-Powered-Mock-Interview-Platform

Status: Active Development

Confidential — Internal Use Only

Contents

Contents	2
1. Product Overview	3
Description	3
2. Objectives and Goals.....	4
3. Target Users	4
4. Actual Problem Statement	4
5. Proposed Solution.....	5
Layer 1 — Database & Access Control.....	5
Layer 2 — The AI Engine & Safe Execution.....	5
Layer 3 — The React Voice & Coding Frontend	5
6. Features.....	5
6.1 F1 — Secure Authentication & Recovery	5
6.2 F2 — Coding Sandbox (Monaco Editor & Test Runs)	6
6.3 F3 — AI Voice Interviewer	6
6.4 F4 — Resume Analyzer (PDF Parse + Tailored Questions).....	6
6.5 F5 — Recruiter Control Panel	6
7. Use Flow	6
7.1 Primary Flow — Candidate Mock Interview Cycle.....	6
7.2 Secondary Flow — Recruiter Review	7
8. Functional Requirements.....	8
9. Non-Functional Requirements.....	9
9.1 Performance	9
9.2 Security.....	9
9.3 Reliability.....	9
9.4 Usability.....	9
9.5 Maintainability.....	9
9.6 Portability.....	10
10. Tech Stack Details	10
Architecture Diagram (Logical)	10
11. Risk Assessment	11

1. Product Overview

One-liner: InterviewAI is an advanced, full-stack mock interview platform that parses candidate resumes and combines them with job templates to generate personalized Technical or HR interview sessions — complete with an AI voice interviewer, a sandboxed coding compiler, and a detailed recruiter dashboard.

Field	Detail
Product Name	InterviewAI (AI-Powered Mock Interview Platform)
Repository	ayushsharma121996-sys/AI-Powered-Mock-Interview-Platform
Owner / Team	Ayush Sharma
Date	July 2026
Version	1.0
Primary Stack	React, TypeScript, Node.js (Express), Prisma ORM, SQLite, Google Gemini API
License	Not specified (Public Repository)

Description

InterviewAI mimics real-life hiring pipelines by providing candidates with realistic technical and behavioral assessments. The application uses a custom-tailored Google Gemini API workflow to analyze uploaded PDF resumes, extract technical skills, and merge them with recruiter-defined job roles. The mock interview screen integrates Text-to-Speech (TTS) questions with a browser-based Speech-to-Text (STT) answer dictation loop. It also hosts a Monaco Editor coding sandbox that executes candidate solutions safely against unit tests using the remote Piston API execution compiler. Recruiters can monitor candidate scores, compare histories, download CSV reviews, and draft customized job templates.

2. Objectives and Goals

1. **Dynamic Customization** — Avoid static, pre-defined question banks. Synthesize targeted Technical and HR questions matching the candidate's exact skills and target job roles.
2. **Multi-Modal Communication** — Provide an auditory voice presence (using SpeechSynthesis) and natural voice answer dictation (using SpeechRecognition) to test spoken clarity and confidence.
3. **Safe Code Execution** — Integrate a secure code editor (Monaco Editor) that runs and checks candidate code (JavaScript, Python, Java) against predefined test suites safely.
4. **Automated AI Grading** — Use generative AI to score each question (0–100) and construct an overall performance card detailing strengths, weaknesses, and a breakdown of communication metrics.
5. **Recruiter Oversight** — Empower recruiting administrators with a candidate leaderboard, comparative scoring analytics, custom templates management, and CSV auditing logs.
6. **Self-Hostable and Zero-Setup** — Rely on a localized SQLite database and Prisma ORM to allow quick local installation with standard Node daemons.

3. Target Users

User Persona	Type	Description
Job Applicants	Primary	Candidates preparing for technical screening rounds, coding challenges, and HR behavioral sessions who need immediate, actionable feedback on their answers.
Recruiters & Hiring Managers	Primary	Staff seeking to create custom roles (e.g., SDE, DevOps), evaluate candidate scoring sheets, compare performance histories, and export assessment records.
Coding Bootcamps & CS Academies	Secondary	Institutions deploying localized testing portals to run mock interview practices for students.
Technical Presenters	Secondary	Developers showing mock integrations of Speech Recognition, remote Piston execution engines, and Google Gemini API structures.

4. Actual Problem Statement

Technical recruitment preparation tools often fail candidates and hiring managers alike due to several persistent design flaws:

7. **Generic Question Pools** — Standard prep tools use hardcoded, generic questions that fail to align with a candidate's actual projects, specific tech stack, or unique resume experience.

8. Lack of Vocal Practice — Most assessment simulators are text-only, failing to recreate the verbal pressure and communication hurdles of actual video/voice interviews where spoken confidence and clear explanations are critical.
9. Improper Code Verification — Generic editors do not compile logic. Candidates write code in a static notepad without real feedback, while platforms that do support compilation require heavy, costly sandbox infrastructure.
10. No Bridge to Recruiters — Self-preparation apps do not share data with recruiters. There is no simple system to allow an administrator to draft custom job requirements (like React, PostgreSQL) and review aggregated candidates' performance.
11. Complex Deployments — Advanced systems require cloud services, Redis instances, or external database accounts, making local prototyping and testing difficult.

5. Proposed Solution

InterviewAI addresses these challenges through a three-layer architecture:

Layer 1 — Database & Access Control

Managed via SQLite (dev.db) and Prisma ORM. Candidate and recruiter profiles have isolated dashboards. Session authentication uses JWT tokens, and an email password reset pipeline is active via Nodemailer (using SMTP or Ethereum mail console fallbacks).

Layer 2 — The AI Engine & Safe Execution

Powered by the Google Gemini API (gemini-1.5-flash) which analyzes raw text extracted from PDF resumes by pdf-parse, evaluates inputs, and generates overall grading feedback. Safe compilation is mediated by wrapping Piston's remote runner API.

Layer 3 — The React Voice & Coding Frontend

A modern dark UI built with Tailwind CSS. It embeds Monaco Editor for coding sessions and controls the Web Speech API to synthesize voice outputs and transcribe incoming verbal replies.

6. Features

6.1 F1 — Secure Authentication & Recovery

- Role Routing — Differentiates candidate view permissions from recruiter panels.
- Password Encryption — Salted and hashed credentials via bcrypt.
- JWT Handshake Verification — Route-guarding middleware checks token validation.
- SMTP Recovery — Forgot/reset password email loop powered by Nodemailer.

6.2 F2 — Coding Sandbox (Monaco Editor & Test Runs)

- VS-style IDE Layout — Embeds Monaco Editor (@monaco-editor/react) supporting JavaScript, Python, and Java templates.
- Safe Sandbox Compilation — Interacts with the EMKC Piston backend execution API to execute custom driver scripts that check algorithms like twoSum, reverseList, and binarySearch.
- Console Outputs — Shows compilation logs, stderr, stdout, and test suite result checklists.

6.3 F3 — AI Voice Interviewer

- Dynamic AI Avatar Panel — Features a glowing, pulsing border animation that changes visually depending on speaking/listening states.
- TTS Narration — Uses the Web Speech API to announce interview questions verbally, with mute/unmute and replay controls.
- STT Transcription — Integrates browser webkitSpeechRecognition to dictate spoken candidate answers directly into text fields.

6.4 F4 — Resume Analyzer (PDF Parse + Tailored Questions)

- PDF Upload — Accepts resume documents, extracts raw text via pdf-parse, and formats skills, education, and projects via Gemini.
- Tailored Question Generation — Combines target role parameters with parsed resume skills to create 5 customized questions (4 concept/behavioral questions and 1 coding puzzle).

6.5 F5 — Recruiter Control Panel

- Candidate Database — Displays candidates, their latest uploaded resumes, interview counts, and average scores.
- Custom Job Templates — Recruiters can create, edit, or delete custom target job templates (roles, descriptions, and required skills).
- CSV Export — Instantly download complete candidate assessments as a formatted CSV spreadsheet.
- Detailed Analytics & Reports — Recruiter view displays individual questions, scores, overall strengths/weaknesses lists, and metrics (accuracy, confidence, grammar, communication, completeness).

7. Use Flow

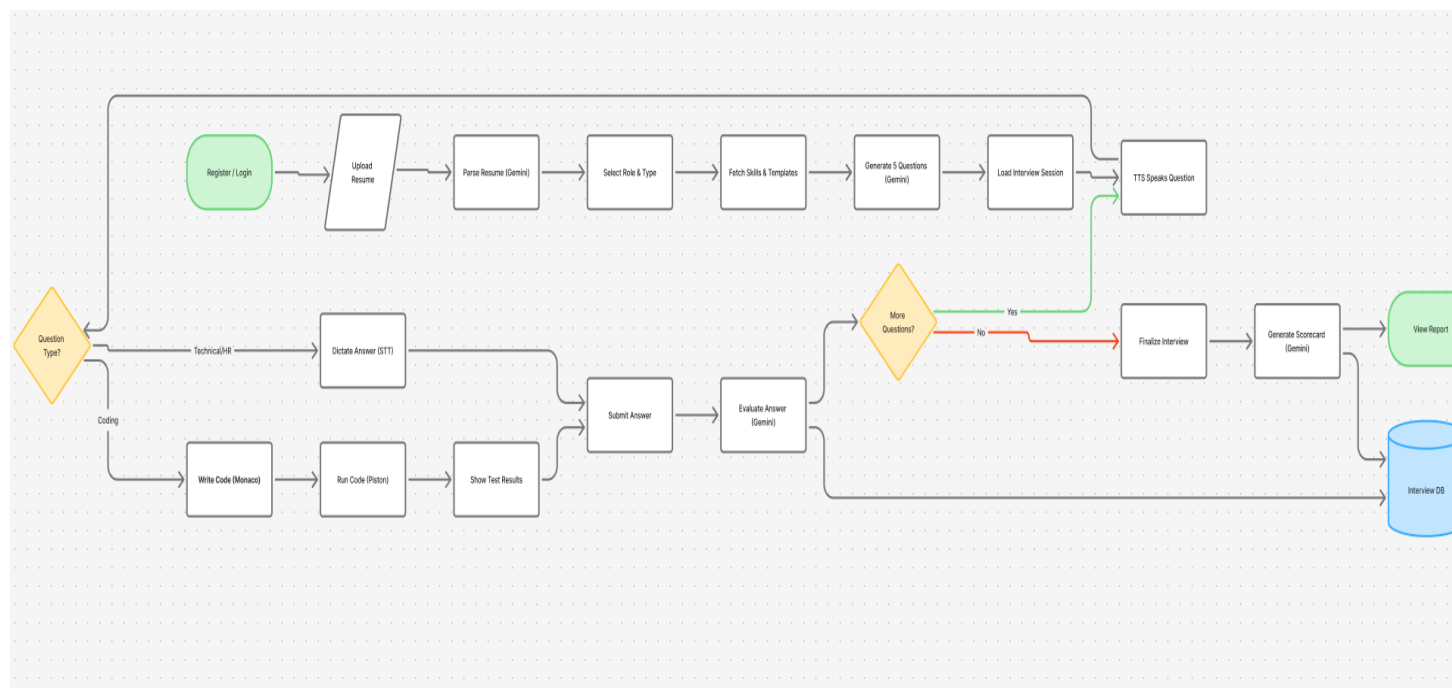
7.1 Primary Flow — Candidate Mock Interview Cycle

12. Register / Login / Upload PDF Resume — Candidate submits credentials and resume to the Express server.
13. Save Credentials & Parsed Data — Server saves credentials and parsed resume data to the SQLite/Prisma database.
14. Start Interview — Candidate selects a role (e.g., SDE) and interview type (Technical/HR).
15. Fetch Context — Server fetches resume skills and recruiter templates matching the role.

16. Request Tailored Questions — Server requests 5 tailored questions from Gemini; Gemini returns a technical/coding questions JSON.
17. Store & Load Session — Server stores the interview session and questions, then loads the session for the candidate.
18. Per-Question Loop (1 to 5) — TTS speaks the question aloud while the avatar glows. For Technical/HR questions, the candidate speaks their answer via voice dictation. For coding questions, the candidate writes logic in Monaco Editor, submits a Run Code request, the server posts the code and driver script to Piston, and Piston returns stdout/stderr and test-pass results rendered as a checklist.
19. Submit Answer — Candidate's answer payload is sent to the server, which requests an evaluation from Gemini against a reference sample.
20. Save Score & Feedback — Gemini returns the individual question score and feedback; the server saves this to the database.
21. Finalise Interview — After all questions, the candidate finalizes the session; the server requests an overall feedback report card from Gemini.
22. Generate Scorecard — Gemini returns the overall score, strengths, weaknesses, and category breakdown; the server updates the interview status to “completed” and saves the scorecard.
23. View Report — Candidate is redirected to /report/:id showing final grades.

7.2 Secondary Flow — Recruiter Review

24. Recruiter logs in; the system detects role privileges and opens the dashboard (/recruiter).
25. Recruiter views all registered candidates, their resume filenames, and their average performance scores.
26. Recruiter reviews candidate scoring metrics or exports the records as a CSV sheet.



Flow chart link-> <https://www.figma.com/board/nQgDelliGOVBLDq5W41eUv/InterviewAI-Candidate-Mock-Interview-Flow?node-id=0-1&t=EbjXvcq6soRuUVd4-1>

8. Functional Requirements

ID	Feature	Requirement Description	Priority
FR-01	Candidate/Recruiter Auth	Secure login, registration, password hashing (bcrypt), and SMTP-based password reset.	High
FR-02	Resume PDF Parser	Upload PDF files, parse content, and extract raw text via memory stream buffers.	High
FR-03	AI Resume Skill Miner	Analyze raw resume text via Gemini API, return skills array, and save details in DB.	High
FR-04	Target Template Merge	Merge candidate resume skills with recruiter templates, defaulting to standard SDE skills if empty.	High
FR-05	Question Synthesis	Request 5 tailored interview questions from Gemini API matching job role and skills.	High
FR-06	Text-To-Speech (TTS)	Narrate questions verbally via browser speech synthesizer with volume controls.	Medium
FR-07	Speech-To-Text (STT)	Dictate candidates' spoken answers to text using built-in speech recognition modules.	Medium
FR-08	Monaco Code Workspace	Embed Monaco code workspace for JavaScript, Python, or Java scripts with syntax highlights.	Medium
FR-09	Compiler Sandbox	Run code submissions via Piston engine, execute unit tests, and print run outputs.	Medium
FR-10	Individual Evaluation	Grade user answers instantly via Gemini, returning a score (0-100) and feedback statement.	High
FR-11	Overall Scorecard Synthesis	Compile overall scores, strengths/weaknesses lists, and category metrics (grammar, confidence, etc.).	High
FR-12	Recruiter Dashboards	Display candidates list, average performance summaries, and templates configurations.	Medium

FR-13	CSV Candidate Export	Generate and download candidacy summary data sheets as a formatted CSV.	Low
--------------	-----------------------------	---	------------

9. Non-Functional Requirements

9.1 Performance

- Prompt Processing — Gemini API response formatting for question generation or feedback synthesis must be compiled in under 4 seconds.
- Code Execution — Piston API execution roundtrips must return test checklists within 3 seconds of submittal.
- Audio Responsiveness — Web Speech synthesis and recognition triggers must be immediate upon user action.

9.2 Security

- Encrypted Storage — Enforce bcrypt password cryptography (work factor 10).
- Guarded Routes — Protect API backend routes using JWT verification checks.
- Sandbox Safety — Keep code executions sandboxed remotely to prevent local server command injection.
- Upload Limits — Enforce maximum resume file upload limit of 10MB to prevent denial-of-service attempts.

9.3 Reliability

- AI Failover Mode — Implement robust mock helper functions to serve fallback questions and assessments if the Gemini API encounters rate limits or offline conditions.
- Graceful Database Transactions — Handle cascade deletes cleanly (e.g. deleting user accounts wipes associated resumes and interviews).

9.4 Usability

- Vocal State Cues — Visual avatar pulses clearly identify speaking, listening, and idle behaviors.
- Monaco Syntax Helpers — Ensure auto-indent and basic JS/Python autocomplete configurations are functional.
- Responsive Layouts — Support mobile-friendly profiles, though coding sandboxes require tablet or desktop viewports.

9.5 Maintainability

- Modular Codebase — Separate routing modules, Prisma schemas, database instances, and AI prompt helper libraries.
- TypeScript Types — Define clear interfaces for questions, evaluations, resume details, and server response objects.

9.6 Portability

- Browser Requirements — Voice recognition modules are supported natively in Google Chrome and Microsoft Edge.
- Database Portability — SQLite keeps database files local, avoiding external database software configurations during deployment.

10. Tech Stack Details

Layer	Technology	Purpose
Frontend Library	React (Vite SPA)	Client-side single page application
Styling Framework	Tailwind CSS	Configured with dark-mode color guidelines
IDE Component	Monaco Editor	@monaco-editor/react
Audio & Speech	Web Speech API	SpeechSynthesis and webkitSpeechRecognition
Backend Framework	Node.js + Express	Configured with TypeScript
Database & ORM	SQLite + Prisma ORM	Local, file-based relational storage
AI Engine	Google Generative AI	@google/generative-ai v1.5-flash model
Email System	Nodemailer	Custom SMTP and Ethereal SMTP test links
PDF Utility	Multer + pdf-parse	Resume upload handling and text extraction
Safe Sandbox Runner	Piston Execution API	emkc.org/api/v2/piston/execute

Architecture Diagram (Logical)

[React + Vite Frontend] (Monaco Editor, Web Speech API, Tailwind UI)

↓↑ REST / JWT

Node.js + Express + TypeScript Server (Auth, Coding, Interview,
Recruiter, Resume routes)

↓

↓

↓

[SQLite / Prisma] [Google Gemini API] [Piston Compiler API]

11. Risk Assessment

#	Risk	Likelihood	Impact	Severity	Mitigation Strategy
R1	Code Sandbox Security Abuse — malicious payloads submitted through the coding editor	Medium	High	High	Rely on remote, isolated container executions (Piston API) rather than running code on the host machine.
R2	Gemini API Rate Limits / Failure — question generation or grading requests fail or are throttled	Medium	High	High	Maintain fallback handlers (getMockQuestions, getMockAnswerEvaluation) that continue mock interviews without disruption.
R3	Web Speech Browser Restrictions — SpeechRecognition unsupported outside Chromium browsers	High	Medium	Medium	Warn non-Chromium users (e.g. Firefox/Safari) when SpeechRecognition is unavailable and provide manual text fallback inputs.
R4	Node process execution leaks — long-running or infinite candidate code loops	Low	Medium	Low	Enforce execution limits on the remote compilation requests to prevent infinite user loops.
R5	SMTP Connection issues — password reset emails fail to send	Medium	Low	Low	Integrate automatic console fallbacks that output Ethereum test links if SMTP server connection times out.

Risk Priority Matrix

Likelihood \ Impact	Low Impact	Med Impact	High Impact
High Likelihood	—	R3	—
Medium Likelihood	R5	—	R1, R2
Low Likelihood	—	R4	—